

Lutendo Netshirando

St10105269

Task 1

PROGRAMMING FOR DATA ANALYTICS 2

PDAN8412/w

Github link: https://github.com/LutendoNetshirando/PDAN8412-w_PART1_ST10105269

DataBase link: <https://www.kaggle.com/datasets/raynardj/classic-english-literature-corpus>

Contents

Introduction	4
Dataset Suitability.....	4
Exploratory Data Analysis	9
Dataset Structure	9
Missing Value Analysis.....	9
Duplicates Analysis	9
Author Distribution	10
Text Length Analysis.....	10
Outlier Analysis	11
Dataset Balancing	11
EDA Summary	12
Data Preprocessing and Feature Preparation.....	13
Text Cleaning.....	13
Training and Testing Split	13
Label Encoding.....	13
Text Tokenization	14
Sequence Preparation	14
Preprocessing Summary	14
LSTM Model Development and Training.....	15
Initial Model Training.....	15
Initial Model Evaluation	16
Model Improvement.....	16
Final Model Configuration	17
Model Evaluation and Results	18
Evaluation Metrics	18
Classification Report Interpretation	18
Confusion Matrix Interpretation	19
Training and Validation Accuracy	20
Training and Validation Loss	20

Overall Evaluation.....	21
Discussion	22
Bibliography	24
Figure 1: Dataset	4
Figure 2: Structure and record counts of the source literary dataset.	5
Figure 3: Balanced modelling dataset containing 2,200 observations across 22 authors.	5
Figure 4: missing values.....	6
Figure 5: Duplicate Records.....	6
Figure 6: Author distribution	6
Figure 7: Analysis of text length	7
Figure 8: training and test split	7
Figure 9:Text Tokenisation.....	8
Figure 10:Model Improvement Result	8
Figure 11:Final LSTM Confusion Matrix	8
Figure 12: Source database tables and record counts.	9
Figure 13: missing values	9
Figure 14: Duplicate Records	10
Figure 15: Distribution of literary text samples by author.	10
Figure 16: Distribution of text lengths in the literary dataset.	11
Figure 17:IQR-based analysis of text-length outliers.....	11
Figure 18: Balanced modelling dataset containing 2,200 observations across 22 authors....	12
Figure 19: Example of cleaned literary text after HTML and formatting removal.	13
Figure 20:Training and testing dataset distribution.	13
Figure 21:Numerical encoding of the 22 author classes.	13
Figure 22: Shape of the tokenised training and testing sequences.....	14
Figure 23:Shape of the tokenised training and testing sequences	14
Figure 24:Architecture and parameter summary of the initial LSTM model	15
Figure 25:Initial LSTM training and validation results.....	16
Figure 26:Classification report for the initial LSTM model.	16
Figure 27:Training results for the improved LSTM configuration.....	16
Figure 28:Final LSTM architecture and parameter summary.....	17
Figure 29: Classification report for the final LSTM model	18
Figure 30: Confusion matrix for the final LSTM model	19
Figure 31: Training and validation accuracy of the final LSTM model.....	20
Figure 32:Training and validation loss of the final LSTM model	20

Introduction

The creation and assessment of a Long Short-Term Memory (LSTM) neural network for literary author categorization are presented in this report. The goal is to determine whether written text patterns may be used to differentiate between various writers.

The literary text dataset books.db, which was acquired from Kaggle, is used in the analysis. The analysis was conducted using Python, Apache Spark, TensorFlow/Keras, and Scikit-learn. The LSTM model was developed using TensorFlow/Keras, while Spark was used for data preparation, exploratory analysis, and dataset balancing.

Exploratory data analysis, data cleansing, text preprocessing, feature preparation, model construction, training, evaluation, and model improvement are all part of an organized machine learning workflow. The final results are viewed in terms of the LSTM model's capacity to generalize to literary texts that have never been seen before.

Dataset Suitability

Because the books.db dataset includes literary content linked to recognizable authors, it is appropriate for the suggested LSTM classification task. The goal variable is represented by the author, while the input variable is represented by the text. This presents a supervised learning problem where the model predicts the corresponding author by identifying patterns in literary texts.

```
[7]: db_path = r"C:\Users\luten\books.db"

conn = sqlite3.connect(db_path)

tables = pd.read_sql_query(
    "SELECT name FROM sqlite_master WHERE type='table';",
    conn
)

tables
```

```
[7]:
```

	name
0	book_original
1	text_files
2	book_file
3	authors
4	books

Figure 1: Dataset

Because written language is sequential, the dataset is especially suitable for an LSTM. The way words are arranged and related to one another affects the text's qualities and meaning. Because LSTM networks are made to process sequential data, they can identify patterns that could help differentiate between different writers' writing styles.

10,171 text items from 292 authors were included in the original dataset. Significant variations in the amount of observations per author were found through exploratory analysis. Authors with at least 100 observations were chosen, and 100 samples were taken at random from each chosen author in order to lessen the impact of class imbalance. There were 2,200 observations in the final modeling dataset, which represented 22 authors with equal representation in each class.

	Number of records: 10171					
10]:	text_id	book_id	bookname	author	text	
0	0	943	The Song Of Roland	Nan	\n\n\n\n\n\n\n\n\n\nThe Song of Roland\n\n\n\n\n\n\n\n...	
1	1	54	Aesop's Fables	Aesop	\n\n\n\n\n\n\n\n\n\nAesop's Fables \n\n\n\n\n...	
2	2	681	The Arabian Nights	Nan	\n\n\n\n\n\n\n\n\n\n KHALIFAH THE...	
3	3	681	The Arabian Nights	Nan	\n\n\n\n\n\n\n\n\n\n	
4	4	681	The Arabian Nights	Nan	\n\n\n\n\n\n\n\n\n\n THE FISHE...	

Figure 2: Structure and record counts of the source literary dataset.

```

Balanced records: 2200
+-----+-----+
|author|count|
+-----+-----+
|Alexandre Dumas|100|
|Andrew Lang|100|
|Charles Dickens|100|
|Edith Wharton|100|
|Elizabeth C. Gaskell|100|
|George Borrow|100|
|George Eliot|100|
|H. G. Wells|100|
|Hans C. Andersen|100|
|Henry James|100|
|Jack London|100|
|Jane Austen|100|
|Joseph Conrad|100|
|Jules Verne|100|
|Leo Tolstoy|100|
|Mark Twain|100|
|Miguel De Cervantes|100|
|Robert Louis Stevenson|100|
|Sir Arthur Conan Doyle|100|
|Thomas Hardy|100|
|Walter Scott|100|
|Wilkie Collins|100|
+-----+-----+

```

Figure 3: Balanced modelling dataset containing 2,200 observations across 22 authors.

Because each author contributes an equal number of observations, the balanced dataset offers a better foundation for classification. This lessens the possibility that authors with bigger initial sample sizes will be favoured by the algorithm.

The dataset does, however, have certain drawbacks. The length of literary texts varies greatly, and 1,202 observations over the computed IQR upper border were found in the exploratory analysis. Since these observations reflect legitimate literary texts rather than blatant data-entry errors, they were left in place. Rather, during preprocessing, a preset sequence length was used.

All things considered, the dataset offers many target classes, labeled sequential text, and enough observations to build an LSTM classification model. Therefore, it is thought to be suitable for applying recurrent neural networks to investigate author categorization.

Analysis Plan:

To guaranty that the dataset was well prepared prior to model building, the report was planned as a sequential machine learning procedure.

The structure and quality of the data were first examined through exploratory data analysis. We

looked into missing values, duplicate records, author distribution, and text length. Next, a balanced subset of the dataset was created in order to address the class distribution.

Missing Values

Missing-value analysis is performed to identify incomplete records that could affect the reliability of the classification model. Each variable is checked for null, empty and blank values before further preprocessing.

```
[1]: from pyspark.sql.functions import col, sum as spark_sum, when, trim

missing_values = spark_df.select([
    spark_sum(
        when(
            col(c).isNull() | (trim(col(c).cast("string")) == ""),
            1
        ).otherwise(0)
    ).alias(c)
    for c in spark_df.columns
])

missing_values.show()

+-----+-----+-----+-----+
|text_id|book_id|bookname|author|text|
+-----+-----+-----+-----+
|      0|      0|      0|      0|      0|
+-----+-----+-----+-----+
```

Figure 4: missing values

Duplicate Records

To find repeated observations that can skew the study, duplicate records are looked at. In order to guaranty that the training data comprises separate text samples, duplicate entries are eliminated.

```
total_records = spark_df.count()
unique_records = spark_df.dropDuplicates().count()
duplicate_records = total_records - unique_records

print("Total records:", total_records)
print("Unique records:", unique_records)
print("Duplicate records:", duplicate_records)

Total records: 10171
Unique records: 10171
Duplicate records: 0
```

Figure 5: Duplicate Records

```
author_counts = (
    spark_df
    .groupBy("author")
    .count()
    .orderBy(col("count").desc())
)

author_counts.show(30, truncate=False)

+-----+-----+
|author|count|
+-----+-----+
|Charles Dickens|1063|
|Mark Twain|463|
|Alexandre Dumas|385|
|Thomas Hardy|360|
|Walter Scott|353|
|Leo Tolstoy|307|
|Jack London|293|
|Elizabeth C. Gaskell|256|
|Jane Austen|247|
|Wilkie Collins|246|
|Henry James|243|
|George Elliot|228|
|Robert Louis Stevenson|227|
|Jules Verne|185|
|H. G. Wells|183|
|Andrew Lang|171|
|Sir Arthur Conan Doyle|166|
|Joseph Conrad|150|
|Miguel De Cervantes|133|
|Hans C. Andersen|128|
|Edith Wharton|113|
|George Borrow|109|
|Aristotle|97|
|Nathaniel Hawthorne|94|
|Washington Irving|93|
|Frances Hodgson Burnett|88|
|Herman Melville|88|
|Alexander Hamilton|86|
|John S. Mill|84|
|Edward Gibbon|81|
+-----+-----+
```

Figure 6: Author distribution

Analysis of Text Length

To find observations that are abnormally short or long, text length is analyzed. Sequence preparation and computing requirements for the LSTM model may be impacted by significant variations in text length. In order to comprehend the distribution of text lengths, descriptive statistics will be employed.

```
text_length_df = spark_df.withColumn(
    "text_length",
    length(col("text"))
)

text_length_df.select(
    "text_length"
).summary(
    "count", "mean", "stddev", "min", "25%", "50%", "75%", "max"
).show()
```

summary	text_length
count	10171
mean	43409.17313931767
stddev	79860.12347879642
min	10
25%	13054
50%	21149
75%	35725
max	953994

Figure 7: Analysis of text length

Second, the text was cleaned up by normalizing whitespace, removing HTML formatting, and changing the content to lowercase. An 80:20 split was used to separate the data into training and testing subsets while preserving representation across the author classes.

Train and Test Split

An 80:20 split is used to separate the cleaned dataset into training and testing sets. To maintain the equal representation of each of the 22 writers in both sets, stratification is used. To ensure an objective assessment of the finished model, the test set is kept apart from model training.

```
from sklearn.model_selection import train_test_split

X = balanced_pd["clean_text"]
y = balanced_pd["author"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Training authors:", y_train.nunique())
print("Testing authors:", y_test.nunique())

Training samples: 1760
Testing samples: 440
Training authors: 22
Testing authors: 22
```

Figure 8: training and test split

Third, the text was transformed into numerical sequences using Keras TextVectorization, and the author labels were numerically encoded. To provide the LSTM a consistent input structure, a maximum vocabulary of 20,000 tokens and a sequence length of 500 tokens were used.

Text Tokenisation

Keras `TextVectorization` is used to transform the cleaned text into numerical sequences. To avoid information from the test set affecting the model, the vocabulary is exclusively learned from the training data. Computational complexity is managed by setting a maximum vocabulary size.

```
[33]: import tensorflow as tf
      from tensorflow.keras.layers import TextVectorization

      MAX_VOCAB_SIZE = 20000
      MAX_SEQUENCE_LENGTH = 500

      vectorizer = TextVectorization(
          max_tokens=MAX_VOCAB_SIZE,
          output_mode="int",
          output_sequence_length=MAX_SEQUENCE_LENGTH
      )

      vectorizer.adapt(X_train)

      print("Vocabulary size:", len(vectorizer.get_vocabulary()))

      Vocabulary size: 20000
```

Figure 9:Text Tokenisation

An embedding layer, a 64-unit LSTM layer, dropout layers, and dense classification layers made up the original LSTM model. For training, the sparse categorical cross-entropy loss function and the Adam optimizer were employed. Early stopping was used to minimize overfitting and cut down on pointless training.

Lastly, accuracy, precision, recall, and F1-score were used to assess the model. Additionally, a confusion matrix was created to assess prediction performance at the class level. To see if generalization could be enhanced, a second model configuration was then trained with tweaked dropout and LSTM parameters.

14/14 ————— 3s 161ms/step
 Accuracy: 0.1182
 Precision: 0.1760
 Recall: 0.1182
 F1-score: 0.0744

Figure 10:Model Improvement Result

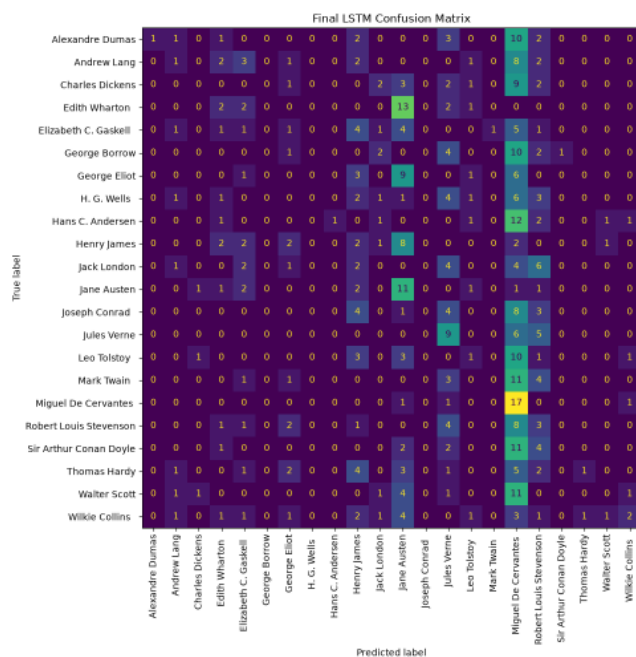


Figure 11:Final LSTM Confusion Matrix

Exploratory Data Analysis

Dataset Structure

Before modelling, the source database was analysed to determine the tables needed for the analysis and to comprehend its structure. Text files, book-to-file linkages, authors, and books were all included in the database.

There were 292 authors, 1,087 books, 10,171 book-file associations, and 10,171 text files in the four pertinent source tables. To create a combined dataset including the text, book name, and author label needed for classification, these tables were connected using the corresponding identifier fields. The structure and record counts from the source database are shown in Figure 12.

```
Authors: 292
Books: 1087
Book-file links: 10171
Text files: 10171
```

Figure 12: Source database tables and record counts.

Missing Value Analysis

To find out if the modeling variables had incomplete observations, a missing-value analysis was performed using Apache Spark. The text identifier, book identifier, book name, author, and text fields were all analyzed.

All five variables had zero missing or blank values in the findings. This suggests that there was no need to remove or impute observations because of missing data in the combined dataset.

Missing Values

Missing-value analysis is performed to identify incomplete records that could affect the reliability of the classification model. Each variable is checked for null, empty and blank values before further preprocessing.

```
[1]: from pyspark.sql.functions import col, sum as spark_sum, when, trim

missing_values = spark_df.select([
    spark_sum(
        when(
            col(c).isNull() | (trim(col(c).cast("string")) == ""),
            1
        ).otherwise(0)
    ).alias(c)
    for c in spark_df.columns
])

missing_values.show()

+-----+-----+-----+-----+-----+
|text_id|book_id|bookname|author|text|
+-----+-----+-----+-----+
|      0|      0|      0|      0|      0|
+-----+-----+-----+-----+

```

Figure 13: missing values

Duplicates Analysis

To find out if repeated observations could affect the model, duplicate records were analysed. The total number of records and the number of unique records were compared using Spark.

There were no duplicate records in the dataset, which had 10,171 total records and 10,171 unique records. As a result, no observations were eliminated due to duplication.

Duplicate Records

To find repeated observations that can skew the study, duplicate records are looked at. In order to guaranty that the training data comprises separate text samples, duplicate entries are eliminated.

```
total_records = spark_df.count()
unique_records = spark_df.dropDuplicates().count()
duplicate_records = total_records - unique_records

print("Total records:", total_records)
print("Unique records:", unique_records)
print("Duplicate records:", duplicate_records)
```

```
Total records: 10171
Unique records: 10171
Duplicate records: 0
```

Figure 14: Duplicate Records

Author Distribution

To determine if the target classes were balanced, the distribution of observations among writers was examined. The original dataset had a significant class imbalance, according to the results. A few of writers had fewer than 200 records, while Charles Dickens had the greatest representation with 1,063 records.

A classification model may favour authors with more observations as a result of this imbalance. As a result, class balance was done prior to model training.

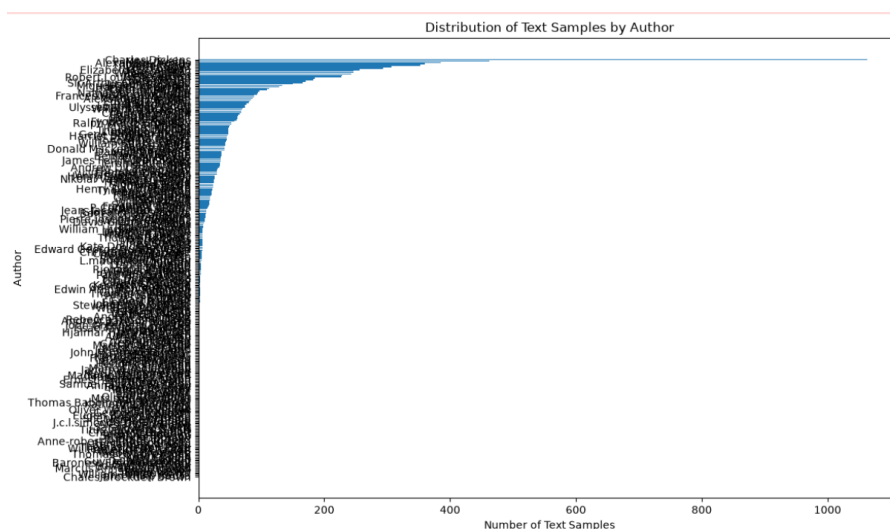


Figure 15: Distribution of literary text samples by author.

The original dataset was very uneven among the author classes, as shown in Figure 15. The adoption of a balanced subset for subsequent modelling was justified by the significant difference between the authors who were most and least represented.

Text Length Analysis

Because LSTM models demand consecutive inputs and significant variation in document length can raise computing costs, text length was examined. The report revealed a median text length of roughly 21,149 characters and a mean text length of roughly 43,409 characters.

Ten characters made up the shortest observation, and 953,994 characters made up the longest. There is a significant variety in the length of the literary writings, as seen by the significant disparity between these numbers.

As a result, during preprocessing, a predetermined sequence length was chosen. This made it

possible to express texts of various lengths using a standard input structure for the LSTM.

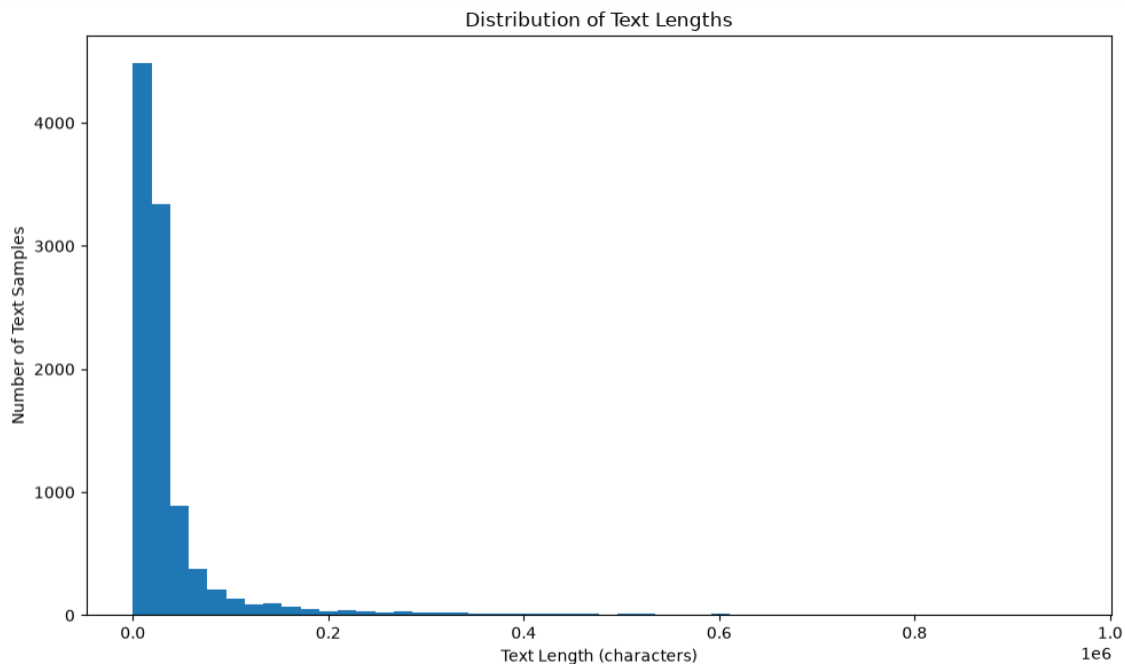


Figure 16: Distribution of text lengths in the literary dataset.

Outlier Analysis

Based on text length, observations that were abnormally long or short were identified using the Interquartile Range (IQR) approach. The IQR was roughly 22,673 characters, with the first quartile being roughly 13,055 characters and the third quartile being roughly 35,727 characters.

The upper limit that was determined was roughly 69,736 characters. Above this threshold, 1,202 observations were found.

Since lengthy literary texts do not always contain inaccurate information, these observations were not immediately eliminated. Rather, in order to maintain the original observations and ensure a consistent input length, sequence padding and truncation were employed during preprocessing.

```
Q1: 13054.5
Q3: 35727.0
IQR: 22672.5
Lower bound: -20954.25
Upper bound: 69735.75
Number of outliers: 1202
```

Figure 17: IQR-based analysis of text-length outliers

Dataset Balancing

Authors with at least 100 observations were chosen in order to lessen the impact of class imbalance. From each qualified author, a maximum of 100 text samples were then chosen at random.

A balanced modelling dataset with 2,200 observations from 22 authors was the result of this process. Every author made exactly 100 observations. Equal representation of the target classes is provided by the balanced structure, which also lessens the possibility that the model will favour authors with higher sample numbers.

Balanced records: 2200

author	count
Alexandre Dumas	100
Andrew Lang	100
Charles Dickens	100
Edith Wharton	100
Elizabeth C. Gaskell	100
George Borrow	100
George Eliot	100
H. G. Wells	100
Hans C. Andersen	100
Henry James	100
Jack London	100
Jane Austen	100
Joseph Conrad	100
Jules Verne	100
Leo Tolstoy	100
Mark Twain	100
Miguel De Cervantes	100
Robert Louis Stevenson	100
Sir Arthur Conan Doyle	100
Thomas Hardy	100
Walter Scott	100
Wilkie Collins	100

Figure 18: Balanced modelling dataset containing 2,200 observations across 22 authors.

EDA Summary

The source dataset included enough labelled text for the suggested classification task, according to the exploratory study. There were no duplicate or missing records found. However, there was a noticeable disparity in text length and a significant class imbalance.

An equal 100-sample representation for 22 writers was created in order to rectify the class imbalance. A predetermined maximum sequence length of 500 tokens was used during sequence preparation to account for the fluctuation in text length.

These preprocessing choices preserved legitimate literary findings while producing a more consistent dataset for later LSTM model development.

Data Preprocessing and Feature Preparation

Text Cleaning

Prior to being fed into the LSTM model, the literary text was cleaned. HTML formatting tags and HTML entities that did not offer useful information for author classification were present in the original text.

Regular expressions were used to eliminate HTML tags, decode HTML entities, and change the content to lowercase. Additionally, excessive whitespace was normalized. These methods preserved the core literary material while reducing unnecessary formatting information.

	author	clean_text
0	Elizabeth C. Gaskell	chapter11 chapter xi first impressions 'there'...
1	Elizabeth C. Gaskell	chapter13 chapter xiii a soft breeze in a sult...
2	Elizabeth C. Gaskell	chapter29 chapter xxix bush-fighting during al...
3	Elizabeth C. Gaskell	chapter27 chapter xxvii gloomy days philip had...
4	Elizabeth C. Gaskell	chapter52 chapter lii squire hamley's sorrow i...

Figure 19: Example of cleaned literary text after HTML and formatting removal.

Training and Testing Split

An 80:20 split was used to separate the balanced dataset into training and testing subsets. To ensure that all 22 authors were represented in both subsets, stratification was used.

There were 440 testing observations and 1,760 training observations in the final dataset. To allow an independent assessment of the model's performance on unseen text, the test dataset was stored apart from model training.

```
Training samples: 1760
Testing samples: 440
Training authors: 22
Testing authors: 22
```

Figure 20: Training and testing dataset distribution.

Label Encoding

Scikit-learn's 'LabelEncoder' was used to translate author names into numerical class labels. Because the neural network works with numerical target values, this conversion was necessary.

A distinct numerical class was given to each of the 22 authors. During LSTM training, the encoded labels were then used as the target values.

```
Number of classes: 22
Classes:
['Alexandre Dumas' 'Andrew Lang' 'Charles Dickens' 'Edith Wharton'
 'Elizabeth C. Gaskell' 'George Borrow' 'George Eliot' 'H. G. Wells'
 'Hans C. Andersen' 'Henry James' 'Jack London' 'Jane Austen'
 'Joseph Conrad' 'Jules Verne' 'Leo Tolstoy' 'Mark Twain'
 'Miguel De Cervantes' 'Robert Louis Stevenson' 'Sir Arthur Conan Doyle'
 'Thomas Hardy' 'Walter Scott' 'Wilkie Collins']
```

Figure 21: Numerical encoding of the 22 author classes.

Text Tokenization

Keras `TextVectorization` was used to transform the cleaned text into numerical representations. To lessen the possibility of information from the test set affecting the model, the vocabulary was solely acquired using the training data.

20,000 tokens was chosen as the maximum vocabulary size. This limited the model's processing requirements while offering a suitably big vocabulary.

Fixed-length number sequences are created from the tokenized text. There are a maximum of 500 tokens in each sequence. To create a uniform input shape for the LSTM model, longer letters are trimmed and shorter texts are padded.

```
X_train_seq = vectorizer(X_train)
X_test_seq = vectorizer(X_test)

print("Training shape:", X_train_seq.shape)
print("Testing shape:", X_test_seq.shape)

Training shape: (1760, 500)
Testing shape: (440, 500)
```

Figure 22: Shape of the tokenised training and testing sequences

Sequence Preparation

For usage as LSTM input, the tokenized text was transformed into fixed-length number sequences. 500 tokens was chosen as the maximum sequence length.

Shorter texts were padded, and texts longer than 500 tokens were terminated. This lessened the computational burden of the significant fluctuation in the original text length and generated a consistent input structure for the neural network.

The training and testing matrices that were produced have $1,760 \times 500$ and 440×500 values, respectively.

Fixed-length number sequences are created from the tokenized text. There are a maximum of 500 tokens in each sequence. To create a uniform input shape for the LSTM model, longer letters are trimmed and shorter texts are padded.

```
X_train_seq = vectorizer(X_train)
X_test_seq = vectorizer(X_test)

print("Training shape:", X_train_seq.shape)
print("Testing shape:", X_test_seq.shape)

Training shape: (1760, 500)
Testing shape: (440, 500)
```

Figure 23: Shape of the tokenised training and testing sequences

Preprocessing Summary

The original literary text was converted into a format appropriate for neural network modelling during the preprocessing phase. The content was normalized, author labels were numerically encoded, HTML artifacts were eliminated, and the text was transformed into fixed-length number sequences.

Stratified data splitting, a training-only vocabulary, and uniform sequence lengths reduced possible sources of data leakage while giving the LSTM model an organized input.

LSTM Model Development and Training

LSTM Architecture

Since the input is sequential natural language data, a Long Short-Term Memory (LSTM) neural network was chosen. An LSTM may find patterns in written text because, in contrast to a traditional feed-forward neural network, it can retain information from prior parts in a sequence.

An embedding layer, a 64-unit LSTM layer, two dropout layers, a 64-unit dense layer, and a final softmax layer with 22 output classes made up the original model. Before the LSTM layer processed the integer-encoded tokens, the embedding layer converted them into dense numerical representations.

The model employed sparse categorical cross-entropy as the loss function and the Adam optimizer. The main training metric was accuracy.

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 500, 128)	2,560,000
lstm_2 (LSTM)	(None, 64)	49,408
dropout_4 (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 64)	4,160
dropout_5 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 22)	1,430

Total params: 2,614,998 (9.98 MB)

Trainable params: 2,614,998 (9.98 MB)

Non-trainable params: 0 (0.00 B)

Figure 24:Architecture and parameter summary of the initial LSTM model

Initial Model Training

The first LSTM was trained with a batch size of 32 for a maximum of 10 epochs. During training, the model's capacity for generalization was tracked using a validation subset. Training could end when validation loss stopped improving thanks to the implementation of early stopping with a patience value of two epochs.

With accuracy rising from 4.97% in the first epoch to 50.14% by epoch six, the original model demonstrated improving training performance. However, validation loss significantly rose while

validation accuracy only attained 12.78%. The model was overfitting the training data, as evidenced by the difference between training and validation performance.

```
Epoch 1/10
44/44 ————— 23s 396ms/step - accuracy: 0.0497 - loss: 3.0919 - val_accuracy: 0.0341 - val_loss: 3.0941
Epoch 2/10
44/44 ————— 21s 478ms/step - accuracy: 0.1009 - loss: 3.0735 - val_accuracy: 0.0568 - val_loss: 3.0906
Epoch 3/10
44/44 ————— 19s 435ms/step - accuracy: 0.1541 - loss: 3.0178 - val_accuracy: 0.0597 - val_loss: 3.0797
Epoch 4/10
44/44 ————— 19s 425ms/step - accuracy: 0.1982 - loss: 2.7147 - val_accuracy: 0.0909 - val_loss: 3.0225
Epoch 5/10
44/44 ————— 19s 430ms/step - accuracy: 0.3189 - loss: 2.1731 - val_accuracy: 0.0966 - val_loss: 3.2635
Epoch 6/10
44/44 ————— 19s 426ms/step - accuracy: 0.5014 - loss: 1.5804 - val_accuracy: 0.1278 - val_loss: 3.4688
```

Figure 25: Initial LSTM training and validation results.

Initial Model Evaluation

The test accuracy of the original model was about 8%. F1-score, recall, and weighted precision were roughly 5%, 8%, and 5%, respectively.

The model's poor test results demonstrated that it was ineffective in generalizing to previously unknown literary texts. For the majority of author classes, the test results remained poor despite a significant rise in training accuracy. This demonstrated the need for model improvement.

```
14/14 ————— 3s 150ms/step
Accuracy: 0.0841
Precision: 0.0511
Recall: 0.0841
F1-score: 0.0543
```

Figure 26: Classification report for the initial LSTM model.

Model Improvement

To address the overfitting found in the original model, two updated configurations were examined.

The first updated model raised dropout from 0.3 to 0.5 and decreased the LSTM layer from 64 to 32 units. The best validation accuracy dropped to 6.82% even though this lessened the gap between training and validation performance. For this reason, the configuration was deemed overly restricted.

A second updated model increased dropout from 0.3 to 0.4 while keeping 64 LSTM units. Because it performed better on untested data, this configuration was chosen as the final model and earned a higher validation accuracy of 14.49%.

```
Epoch 1/10
44/44 ————— 17s 265ms/step - accuracy: 0.0405 - loss: 3.0919 - val_accuracy: 0.0341 - val_loss: 3.0915
Epoch 2/10
44/44 ————— 12s 267ms/step - accuracy: 0.0675 - loss: 3.0824 - val_accuracy: 0.0455 - val_loss: 3.0916
Epoch 3/10
44/44 ————— 13s 303ms/step - accuracy: 0.0724 - loss: 3.0647 - val_accuracy: 0.0540 - val_loss: 3.0856
Epoch 4/10
44/44 ————— 14s 316ms/step - accuracy: 0.0888 - loss: 3.0230 - val_accuracy: 0.0682 - val_loss: 3.0585
Epoch 5/10
44/44 ————— 14s 324ms/step - accuracy: 0.1108 - loss: 2.9011 - val_accuracy: 0.0625 - val_loss: 3.0623
Epoch 6/10
44/44 ————— 15s 335ms/step - accuracy: 0.1605 - loss: 2.7166 - val_accuracy: 0.0682 - val_loss: 3.0798
```

Figure 27: Training results for the improved LSTM configuration.

Final Model Configuration

A vocabulary of 20,000 tokens and a maximum sequence length of 500 tokens were used in the final LSTM configuration. An LSTM layer with 64 units came after the embedding dimension of 128.

In order to provide further regularization, dropout was raised to 0.4. Prior to the final 22-class softmax layer, a thick layer comprising 64 units with ReLU activation was employed.

The Adam optimizer with sparse categorical cross-entropy loss was used to train the model. The model weights linked to the best validation loss were restored by early stopping.

```
Epoch 1/10
44/44 ————— 21s 376ms/step - accuracy: 0.0419 - loss: 3.0928 - val_accuracy: 0.0625 - val_loss: 3.0902
Epoch 2/10
44/44 ————— 19s 429ms/step - accuracy: 0.0703 - loss: 3.0873 - val_accuracy: 0.0455 - val_loss: 3.0904
Epoch 3/10
44/44 ————— 20s 455ms/step - accuracy: 0.0902 - loss: 3.0699 - val_accuracy: 0.0625 - val_loss: 3.0808
Epoch 4/10
44/44 ————— 21s 464ms/step - accuracy: 0.1349 - loss: 3.0034 - val_accuracy: 0.0540 - val_loss: 3.0155
Epoch 5/10
44/44 ————— 22s 500ms/step - accuracy: 0.1477 - loss: 2.7924 - val_accuracy: 0.0909 - val_loss: 2.9691
Epoch 6/10
44/44 ————— 23s 512ms/step - accuracy: 0.2244 - loss: 2.4686 - val_accuracy: 0.1023 - val_loss: 2.9182
Epoch 7/10
44/44 ————— 24s 533ms/step - accuracy: 0.3636 - loss: 2.0217 - val_accuracy: 0.1193 - val_loss: 3.0392
Epoch 8/10
44/44 ————— 22s 492ms/step - accuracy: 0.4787 - loss: 1.6226 - val_accuracy: 0.1449 - val_loss: 3.2773
```

Figure 28: Final LSTM architecture and parameter summary

Model Evaluation and Results

Evaluation Metrics

The unseen test dataset with 440 observations was used to assess the final model. Accuracy, precision, recall, and F1-score were the four categorization measures employed.

The percentage of correctly identified observations is a measure of accuracy. Recall quantifies the percentage of real data that were properly identified, whereas precision quantifies the percentage of predicted classes that were accurate. A combined measure of recall and precision is offered by the F1-score.

Because author-level performance differed among the 22 classes, these metrics offer a more comprehensive evaluation of categorization performance than accuracy alone.

Metric	Initial LSTM	Final LSTM
Accuracy	8.00%	11.82%
Precision	5.00%	17.60%
Recall	8.00%	11.82%
F1-score	5.00%	7.44%

Table 1: Final Model Performance

On the test dataset, the final LSTM's accuracy was 11.82%, while the original model's accuracy was 8.00%. While recall rose from 8.00% to 11.82%, precision climbed from roughly 5.00% to 17.60%. Additionally, the F1-score increased from roughly 5.00% to 7.44%.

The aggregate results are still low, even if the improvement shows that the updated design produced greater performance. Random categorization would yield a baseline accuracy of roughly 4.55% because there are 22 equally represented authors in the dataset. As a result, the final model outperformed this baseline, although its predictive power is still restricted.

	precision	recall	f1-score	support
Alexandre Dumas	1.00	0.05	0.10	20
Andrew Lang	0.12	0.05	0.07	20
Charles Dickens	0.00	0.00	0.00	20
Edith Wharton	0.14	0.10	0.12	20
Elizabeth C. Gaskell	0.06	0.05	0.05	20
George Borrow	0.00	0.00	0.00	20
George Eliot	0.00	0.00	0.00	20
H. G. Wells	0.00	0.00	0.00	20
Hans C. Andersen	1.00	0.05	0.10	20
Henry James	0.06	0.10	0.08	20
Jack London	0.00	0.00	0.00	20
Jane Austen	0.16	0.55	0.25	20
Joseph Conrad	0.00	0.00	0.00	20
Jules Verne	0.20	0.45	0.28	20
Leo Tolstoy	0.11	0.05	0.07	20
Mark Twain	0.00	0.00	0.00	20
Miguel De Cervantes	0.10	0.85	0.19	20
Robert Louis Stevenson	0.07	0.15	0.09	20
Sir Arthur Conan Doyle	0.00	0.00	0.00	20
Thomas Hardy	0.50	0.05	0.09	20
Walter Scott	0.00	0.00	0.00	20
Wilkie Collins	0.33	0.10	0.15	20
accuracy			0.12	440
macro avg	0.18	0.12	0.07	440
weighted avg	0.18	0.12	0.07	440

Figure 29: Classification report for the final LSTM model

Classification Report Interpretation

The classification report reveals a wide range of author performance. At 85%, Miguel De Cervantes

had the highest recall, followed by Jane Austen at 55% and Jules Verne at 45%. These findings suggest that certain author-specific patterns were more successfully recognized by the model than others.

However, a few of classes showed zero recall, suggesting that the model was unable to accurately identify observations that belonged to those authors. Performance was uneven among the author classes, as evidenced by the macro-average F1-score of 0.07.

The findings imply that while the model picked up certain distinctive patterns, it was unable to apply these patterns consistently to all 22 writers.

Confusion Matrix Interpretation

The model's predictions for each of the 22 author classifications are shown graphically in the confusion matrix. While data outside the diagonal indicate incorrect classifications, accurate predictions are shown along the diagonal.

The matrix shows that instead of being consistently focused on the right classes, predictions were scattered among several author classifications. This corroborates the classification report, which indicated that a few authors performed well while a number of others performed poorly.

The failure of the current LSTM configuration to distinguish the writing styles of all writers is indicated by the confusion between author classes.

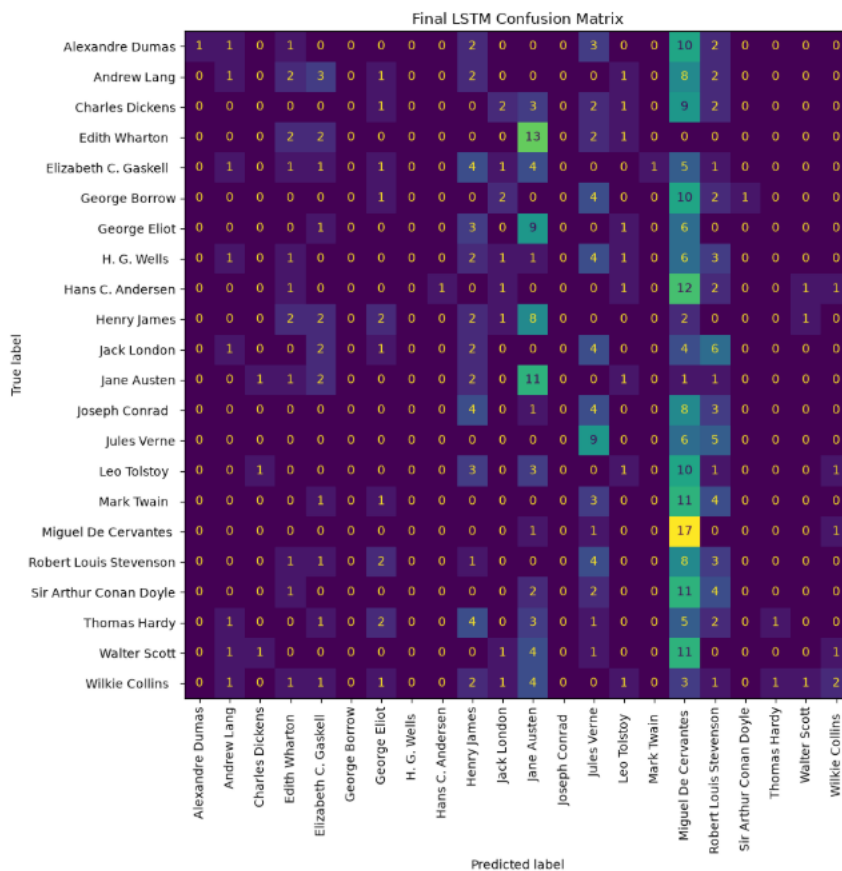


Figure 30: Confusion matrix for the final LSTM model

Training and Validation Accuracy

The model's performance on training and validation data differs significantly, as seen by the training and validation accuracy curves. Later epochs saw a significant rise in training accuracy, while validation accuracy stayed quite low.

The model was learning patterns unique to the training observations without attaining comparable performance on unknown observations, as seen by this divergence. The outcome explains the comparatively low test accuracy and is consistent with overfitting.

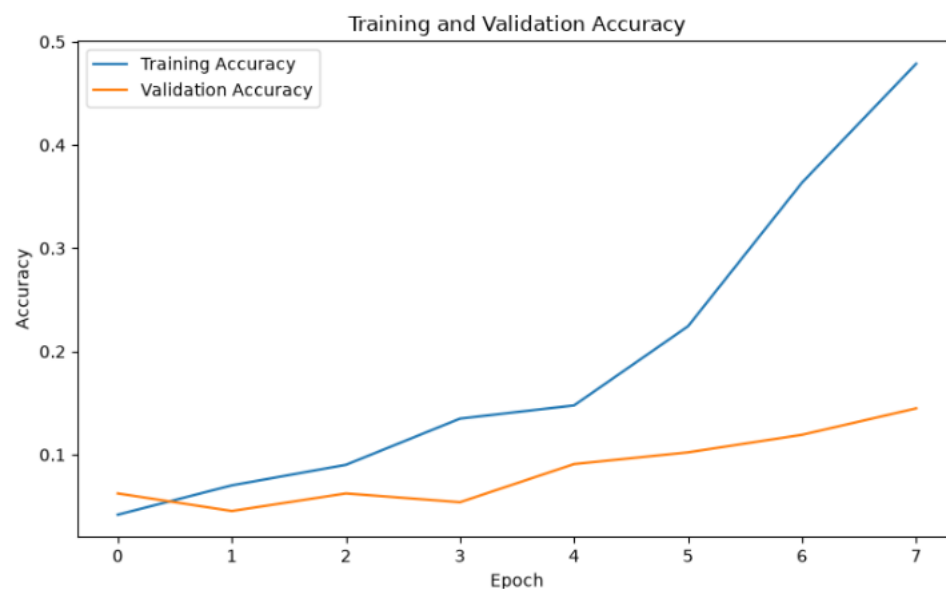


Figure 31: Training and validation accuracy of the final LSTM model.

Training and Validation Loss

Additional proof of overfitting is provided by the loss curves. Validation loss gradually grew, whereas training loss dropped as the model learned the training data. The rise in validation loss suggests that further training did not enhance generalization.

Because early stopping ended training once validation performance stopped improving and restored the model weights from the best validation point, it was therefore suitable.

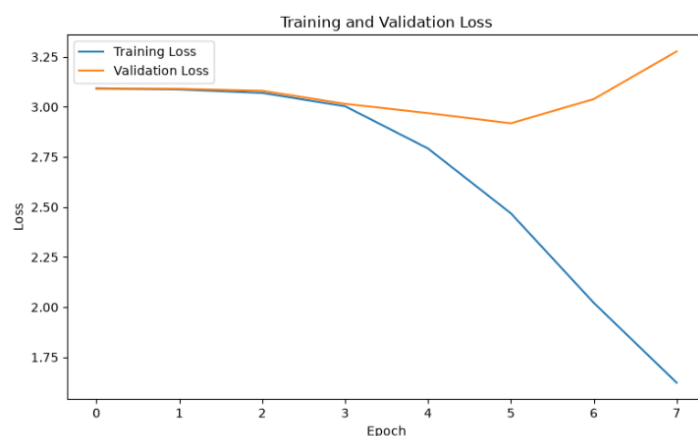


Figure 32: Training and validation loss of the final LSTM model

Overall Evaluation

With test accuracy rising from 8.00% to 11.82%, the final LSTM outperformed the original configuration overall. The improvement shows how important it is to change the regularization parameters and model design.

The final model did, however, nevertheless show some modest generalization. The low F1-score and the significant difference between training and validation performance suggest that additional optimization would be necessary to obtain accurate author categorization.

Increasing the quantity of training data, experimenting with various sequence lengths and designs, employing advanced natural language processing algorithms, and splitting at the book level rather than the text level to minimize potential data leakage are all potential enhancements.

Discussion

The findings show that while an LSTM may learn some patterns related to literary writers, the classification process is still difficult when several authors are taken into account at once. The 22 chosen authors were equally represented in the balanced sample, which lessened the impact of the initial class imbalance.

The test accuracy of the original model was 8.00%. The final model scored 11.82% test accuracy after increasing dropout while keeping 64 LSTM units. The ultimate performance is still comparatively low even though this is an improvement. A random classifier would achieve about 4.55% accuracy with 22 evenly represented classes; therefore, the final model outperformed this baseline but did not reach high predictive performance.

Overfitting was also shown in the training results. Later epochs saw an increase in validation loss and a significant improvement in training accuracy while validation accuracy remained significantly lower. This implies that the model did not acquire adequately generalizable representations of author writing styles, instead learning patterns unique to the training observations.

The classification report also showed that different writers performed differently. While other writers showed no recall, Miguel De Cervantes, Jane Austen, and Jules Verne had rather strong recall. This suggests that the model found some author-specific patterns more easily than others. According to the results, the chosen LSTM architecture offers an effective demonstration of recurrent neural network text classification, however, further optimization would be necessary for a production quality author identification system.

Limitations

When interpreting the results, a number of restrictions should be taken into account.

First, there was a significant class imbalance in the original dataset. Although a balanced dataset was created to remedy this, balancing only made 2,200 samples of the original dataset's observations available.

Second, the length of the literary writings varied significantly. As a result, a maximum sequence length of 500 tokens was employed. Although this method makes the input computationally manageable, it may shorten lengthy texts to remove information.

Third, overfitting was seen in the model. The learned representations did not consistently generalize across all author classes, as indicated by the comparatively poor test results.

Lastly, the text record level was used for the present train test split. Similar content may appear in both training and testing data when several text segments come from the same book. A more robust test of the model's ability to recognize an author from truly unseen volumes might be possible with a book level split.

Conclusion

In order to predict literary authors from written text, this analysis created an LSTM-based text categorization model. A full machine learning workflow, including data exploration, cleaning, balancing, text tokenization, sequence preparation, model training, evaluation, and model improvement, was followed in the analysis.

Data preparation, exploratory analysis, and balancing were done using Spark, the LSTM model was developed with TensorFlow/Keras, and data splitting, label encoding, and evaluation were done with Scikit-learn.

The test accuracy of the original model was 8.00%. The resulting model had an accuracy of 11.82%, weighted precision of 17.60%, recall of 11.82%, and an F1-score of 7.44% after the model configuration was changed. The results show that the model had limited generalization and continued to experience overfitting, despite the new model's improvement over the original configuration.

Overall, the study shows how LSTM networks may be used for sequential text categorization and emphasizes the significance of class balancing, exploratory analysis, suitable preprocessing, model evaluation, and hyperparameter tuning. Larger training datasets, different sequence lengths, book-level data splitting, and more advanced natural language processing architectures could all be the subject of future research.

Bibliography

(Raynard), X. Z., 2020. *Classic English Literature Corpus and Meta Data*. [Online]

Available at: <https://www.kaggle.com/datasets/raynardj/classic-english-literature-corpus>

[Accessed 14 September 2026].

scikit-learn, n.d. *scikit-learn Machine Learning in Python*. [Online]

Available at: <https://scikit-learn.org/stable/>

[Accessed 14 September 2026].

Spark, A., n.d. *Apache Spark - A Unified engine for large-scale data analytics*. [Online]

Available at: <https://spark.apache.org/docs/latest/>

[Accessed 14 September 2026].

tensorflow, n.d. *An end-to-end platform for machine learning*. [Online]

Available at: <https://www.tensorflow.org/>

[Accessed 14 September 2026].